

Semi-Supervised Learning

Complete Theory, Assumptions, Methods, Applications & Exam Traps

1. What is Semi-Supervised Learning?

Definition:

- Machine learning paradigm using **both labeled and unlabeled** data simultaneously
- Bridges gap between supervised (100% labeled) and unsupervised (0% labeled)
- Leverages abundant cheap unlabeled data to compensate for scarce expensive labeled data

Core Philosophy:

- **Supervised:** Learns mapping $f : X \rightarrow Y$ from labeled examples $\{(x_i, y_i)\}_{i=1}^l$
- **Unsupervised:** Discovers structure in unlabeled data $\{x_i\}_{i=1}^{l+u}$ without y
- **Semi-Supervised:** Uses both $\{(x_i, y_i)\}_{i=1}^l$ and $\{x_i\}_{i=l+1}^{l+u}$ where $l \ll u$

Key Insight:

- Unlabeled data reveals underlying **data distribution** and **manifold structure**
- This structure acts as a **regularizer** guiding classification boundaries
- Properly leveraged, yields better generalization than using either alone

Exam Trap: SSL is NOT supervised + unsupervised combined sequentially; it's an integrated paradigm.

2. When to Use SSL - Decision Framework

Decision Matrix:

Scenario	Labels Available	Recommended
Abundant labels, clean data	l large, high quality	Supervised Learning
No labels, exploratory analysis	$l = 0$	Unsupervised Learning
Few labels, abundant unlabeled	l small, u large	Semi-Supervised
Labels but unlabeled different distribution	l available, u from different $P(x)$	Avoid SSL

Key Criteria for SSL Suitability:

- 1 Labeled data is **scarce** and **expensive** to obtain
- 2 Unlabeled data is **abundant** and **cheap**
- 3 Labeled and unlabeled data from **same distribution** $P(x)$
- 4 **Assumptions** (clustering, smoothness, manifold) are reasonably satisfied
- 5 Business need for **generalizable model** (inductive) OR just current labeling (transductive)

Exam Trap: SSL does NOT help if labeled/unlabeled distributions differ (dataset shift).

3. Supervised vs Unsupervised vs SSL

Detailed Comparison:

Aspect	Supervised	Unsupervised	Semi-Supervised
Training Data	$\{(x_i, y_i)\}_{i=1}^l$	$\{x_i\}_{i=1}^n$	$\{(x_i, y_i)\}_{i=1}^l + \{x_i\}_{i=l+1}^{l+u}$
Goal	Predict y for new x	Find structure in x	Predict y using structure
Data Labels	Required for all	None required	Required for subset
Typical Use	Classification/Regression	Clustering/Dim Reduction	Both
Label Cost	High (all labeled)	Zero	Low (few labeled)
Performance with few labels	Poor (overfitting)	N/A	Good (with assumptions)
Representation Learning	From labels only	From data only	From both

Key Takeaway:

- SSL optimally combines **supervisory signal** from labels with **structural information** from unlabeled data
- The unlabeled data acts as a **regularizer** preventing overfitting

4. Real-World Application Domains

Fraud Detection (Credit Card Company):

- I = few confirmed fraud cases (expensive expert review)
- u = millions of transactions (cheap data)
- **Outcome:** 30% reduction in false positives, better fraud detection

Medical Imaging:

- I = few radiologist-annotated scans (expertise scarce)
- u = thousands of unlabeled scans (abundant)
- Models learn from unlabeled to improve diagnosis

Natural Language Processing:

- I = few labeled documents (manual annotation expensive)
- u = millions of web documents (unlabeled)
- Co-training with multiple views (text + metadata)

Risk Management:

- I = expert-classified operational incidents (human error, system failure, external event)
- u = large database of incident reports (unstructured text)
- Improved risk categorization

Network Security:

- I = confirmed malicious/benign traffic (forensic analysis)
- u = massive network logs (continuous stream)
- Better detection of novel threats

5. Why Labels are Scarce & Expensive

Reasons for Label Scarcity:

1 Expertise Requirement:

- Medical diagnosis requires trained radiologists
- Fraud detection needs forensic accountants
- Risk classification needs domain specialists

2 Time Cost:

- Manual review of each instance takes minutes to hours
- Millions of instances → impossible

3 Financial Cost:

- Expert time = high hourly rate
- Large-scale labeling = prohibitively expensive

4 Data Sensitivity:

- Privacy constraints prevent sharing labels
- Regulatory restrictions on data usage

5 Event Rarity:

- Fraud cases are rare (0.1% of transactions)
- Default events in loans are uncommon
- Positive class is rare, making labels scarce

Exam Trap: SSL is NOT about replacing labels entirely; it's about maximizing value from limited labels.

6. The Two Naive Approaches & Why They Fail

Approach A: Discard Labels, Use Unsupervised

- **Process:** Apply clustering (K-means, DBSCAN) to all data, ignore labels
- **Problem:** Natural clusters may NOT correspond to classes
- **Example:** Clustering fraud data by transaction amount may group by size, not fraud status
- **Consequence:** Throws away human expertise, valuable labels

Approach B: Discard Unlabeled, Use Supervised

- **Process:** Train classifier only on labeled data / instances
- **Problem:** l is too small $\Rightarrow$ overfitting, poor generalization
- **Example:** 100 labeled fraud cases among 1M transactions $\Rightarrow$ model sees limited patterns
- **Consequence:** Ignores structural information in abundant unlabeled data

Approach C: Manually Label Everything

- **Process:** Label all n instances with experts
- **Problem:** Prohibitively expensive, time-consuming, impossible at scale
- **Consequence:** Not feasible in practice

SSL is the optimal solution: Uses BOTH sources of information optimally.

7. Assumptions Overview - Mathematical Formulation

Why Assumptions are Critical:

- SSL only works if unlabeled data distribution $P(x)$ relates to conditional $P(y|x)$
- If no relationship, unlabeled data provides no information about classification
- **FALSE ASSUMPTION:** Unlabeled data always helps (INCORRECT!)

Three Core Assumptions:

- 1 **Clustering Assumption:** Data forms distinct clusters; each cluster $\approx$ one class

$$\text{If } x_i, x_j \text{ in same cluster, then } P(y|x_i) \approx P(y|x_j)$$

- 2 **Smoothness Assumption:** Close in feature space $\Rightarrow$ same label

$$x_i \approx x_j \implies y_i \approx y_j \quad (\text{local consistency})$$

- 3 **Manifold Assumption:** High-dimensional data lies on low-dimensional manifold

$$\text{Data} \subset \mathcal{M} \subset \mathbb{R}^D, \quad \dim(\mathcal{M}) = d \ll D$$

Labels vary smoothly along $\mathcal{M}$

Mathematical Connection:

- All three imply: $P(x)$ contains information about $P(y|x)$
- SSL uses $P(x)$ (from unlabeled data) to better estimate $P(y|x)$

8. Assumption 1: Clustering - Detailed

Formal Statement:

- The data is composed of k distinct clusters $C_1, C_2, \dots, C_k$
- Each cluster corresponds to a single class $c \in \{1, \dots, C\}$
- Clusters are separated by **low-density regions** where $P(x)$ is near zero
- The decision boundary should lie in these low-density regions

Mathematical Formulation:

$$P(x) \approx \sum_{j=1}^k \pi_j \mathcal{N}(x; \mu_j, \Sigma_j)$$

where each component $\mathcal{N}$ corresponds to one class

Example - Fraud Detection:

- Fraudulent transactions: C_1 with features (small amounts, odd locations, unusual time)
- Benign transactions: C_2 with features (normal patterns, consistent behavior)
- C_1 and C_2 are well-separated in feature space

If Violated:

- Classes intermingled, no separation
- Example: Fraud and normal transactions have identical feature distributions
- Unlabeled data provides NO information about class boundaries

9. Assumption 2: Smoothness - Detailed

Formal Statement:

- If two points x_i and x_j are close in the feature space (under some metric), they should have the same label $y_i = y_j$
- The decision boundary should pass through regions where $P(x)$ is low (low density)
- This is also called the "**Local Consistency**" assumption

Mathematical Formulation:

$$\forall x_i, x_j \text{ with } d(x_i, x_j) < \epsilon \implies P(y|x_i) \approx P(y|x_j)$$

The decision boundary satisfies:

$$\text{Boundary} \subset \{x : P(x) \text{ is small}\}$$

Example - Network Traffic:

- Similar packet sizes, frequencies, ports $\implies$ same classification (malicious/benign)
- Boundary between classes lies in sparse regions of feature space
- No boundary cuts through dense cluster of packets

If Violated:

- Boundary cuts through dense region
- Close points have different labels
- Example: Very similar network traffic patterns classified differently

10. Assumption 3: Manifold - Detailed

Formal Statement:

- High-dimensional data $\mathcal{X} \subset \mathbb{R}^D$ lies on or near a low-dimensional manifold $\mathcal{M}$
- $\dim(\mathcal{M}) = d \ll D$
- Labels vary **smoothly** along $\mathcal{M}$
- Points on same connected component of $\mathcal{M}$ likely same class

Mathematical Formulation:

$\mathcal{X} \subset \mathcal{M} \subset \mathbb{R}^D$, where $\mathcal{M}$ is a d -dimensional Riemannian manifold

If $x_1, x_2 \in \mathcal{M}$ are connected, then $P(y|x_1) \approx P(y|x_2)$

Example - Face Images:

- Input: 100×100 pixels = 10,000 dimensions ($D = 10,000$)
- Actual variation: pose (2D), expression (1D), lighting (2D) $\Rightarrow d \approx 5$
- Manifold $\mathcal{M}$ captures these variations
- Faces of same person lie on same connected manifold

Why It Matters:

- Unlabeled data maps the manifold structure
- Labels guide classification on the manifold
- Result: Accurate classification with few labels

11. Assumptions - Visual Violations

Violation 1: No Clusters (Uniform Distribution)

- Data uniformly spread in feature space
- No dense regions, no separation
- Unlabeled data gives no information about class boundaries
- **Result:** SSL fails, unlabeled data not helpful

Violation 2: Overlapping Clusters

- Clusters partially overlap
- Boundary must cut through dense region
- Smoothness assumption violated
- **Result:** SSL misplaces boundary, performance degrades

Violation 3: Manifold Mixes Classes

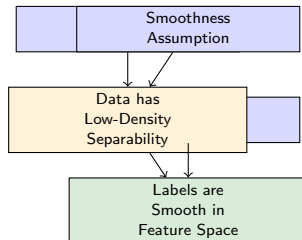
- Different classes on same manifold
- Structure doesn't correspond to labels
- Example: Two people with similar pose/lighting on same manifold
- **Result:** Unlabeled data maps manifold but labels don't vary smoothly

Violation 4: Dataset Shift

- Labeled data distribution $P_l(x) \neq$ unlabeled $P_u(x)$
- Structure learned from $P_u(x)$ doesn't apply to classification
- **Result:** SSL makes performance worse

12. Assumptions - Mathematical Connections

Relationship Between Assumptions:



Key Connections:

- 1 Clustering $\implies$ Smoothness: Clusters separated by low-density regions $\implies$ boundaries in low density
- 2 Manifold $\implies$ Smoothness: If labels vary smoothly on manifold, close points on manifold have same label
- 3 All $\implies$: Unlabeled data $\{x_i\}_{i=l+1}^{l+u}$ reveals structure of $P(x)$ which guides $P(y|x)$

Exam Trap: Violation of ANY assumption can cause SSL to fail.

13. When Assumptions are Satisfied - Success

Example: Document Classification

Data:

- $I = 100$ labeled documents (topics: Sports, Politics, Technology)
- $u = 100,000$ unlabeled documents
- Features: TF-IDF word frequencies (high-dimensional)

Assumptions Satisfied:

- 1 **Clustering**: Documents on same topic use similar vocabulary $\Rightarrow$ form clusters
- 2 **Smoothness**: Similar documents (cosine distance) have same topic
- 3 **Manifold**: Documents lie on topic-specific low-dimensional subspace

Result:

- SSL accurately classifies unlabeled documents
- Outperforms supervised-only (100 labels) and unsupervised-only
- Unlabeled data maps topic-specific vocabulary manifolds

Another Example: Handwritten Digit Recognition

- 28x28 pixel images ($D = 784$)
- 10 classes (0-9)
- $I = 100$ labeled digits, $u = 60,000$ unlabeled
- Digits of same class lie on same manifold (variations in writing style)

14. When Assumptions are Violated - Failure

Example: Facial Recognition with Varying Pose

Data:

- $l = 50$ labeled face images (Person A, Person B)
- $u = 10,000$ unlabeled face images
- Features: Pixel values ($D = 10,000$)

Problem 1 - Manifold Mixing:

- Person A at extreme pose may look like Person B at neutral pose
- Same manifold may contain different people
- **Manifold Assumption Violated**

Problem 2 - No Clusters:

- Face images of different people intermingled in pixel space
- **Clustering Assumption Violated**

Problem 3 - Dataset Shift:

- Labeled images: Professional studio photos
- Unlabeled images: Web-crawled photos
- Different lighting, background, resolution
- **Distribution Mismatch**

Result:

- SSL performance degrades below supervised-only
- Pseudo-labels are incorrect $\Rightarrow$ error amplification
- **SSL is not a magic solution; assumptions matter!**

15. Inductive vs Transductive - Core Concept

Key Distinction at Highest Level:

Inductive	Transductive
Learns a generalizable model	Only labels current unlabeled data
Produces function $f : X \rightarrow Y$	No function; just assigns labels
Can classify new, unseen data	Cannot handle new data
Preferred for production/deployment	Limited to offline analysis
Examples: Self-training, Co-training	Example: Label Propagation
"Open world" view	"Closed world" view

Mathematical Formulation:

- **Inductive:** $\hat{f} = \arg \min_{f \in \mathcal{H}} \mathcal{L}(f, \{(x_i, y_i)\}_{i=1}^l) + \lambda \mathcal{R}(f, \{x_i\}_{i=l+1}^{l+u})$
- **Transductive:** $\hat{y}_{l+1}, \dots, \hat{y}_{l+u} = \arg \max P(y_{l+1}, \dots, y_{l+u} | \{(x_i, y_i)\}_{i=1}^l, \{x_i\}_{i=l+1}^{l+u})$

16. Transductive Methods - Detailed

Label Propagation (Graph-Based):

Mathematical Framework:

1 Build graph $G = (V, E)$ where $V = \{x_1, \dots, x_{l+u}\}$

2 Edge weights $w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$

3 Labels propagate through graph:

$$Y^{(t+1)} = PY^{(t)}$$

where P is row-normalized transition matrix

4 Fixed point: $Y^* = \lim_{t \rightarrow \infty} Y^{(t)}$

Properties:

- Labels from labeled nodes "flow" to unlabeled neighbors
- High confidence when surrounded by similar labeled nodes
- No model f learned; just node labels Y^*

Limitations:

- 1 **No generalizable model:** Can't classify new x_{l+u+1}
- 2 **Computational:** Graph with n nodes $\Rightarrow O(n^2)$ for dense graph
- 3 **Sensitive:** Choice of σ and graph structure matters

When to Use:

- Only need to label current dataset (offline analysis)
- No need for future predictions
- Small dataset where graph construction is feasible

17. Inductive Methods - Detailed

Core Idea:

- 1 Learn a function $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$ from both labeled and unlabeled data
- 2 Can be applied to any new $x \notin \{x_1, \dots, x_{l+u}\}$
- 3 Preferred in practice because:
 - 1 Deployable in production
 - 2 Can score new instances in real-time
 - 3 Adapts as new data arrives

General Framework:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \underbrace{\sum_{i=1}^l \ell(f(x_i), y_i)}_{\text{Supervised Loss}} + \underbrace{\lambda_1 \sum_{i=l+1}^{l+u} \mathcal{R}(f, x_i)}_{\text{Unsupervised Regularizer}} + \underbrace{\lambda_2 \|f\|_{\mathcal{H}}^2}_{\text{Model Complexity}}$$

Unsupervised Regularizers:

- 1 Low-Density Separation: f should be confident on unlabeled data
- 2 Manifold Regularization: f should be smooth on the manifold
- 3 Entropy Minimization: Minimize $-\sum_{x \in \mathcal{U}} P(y|x) \log P(y|x)$

Main Approaches:

- 1 Wrapper methods (Self-training, Co-training)
- 2 Unsupervised pre-processing (PCA, Autoencoders, Cluster-then-Label)

18. Inductive vs Transductive - Decision Framework

Decision Matrix for Choosing:

Requirement	Inductive	Transductive
Need to classify future data	YES	NO
Real-time scoring required	YES	NO
Model deployment needed	YES	NO
Only current dataset matters	May be overkill	YES
Offline analysis only	Can be used	YES
Computational constraints	More complex	Simpler
Need interpretable model	YES	NO

Recommendation:

- **Production/Deployment:** Always use Inductive
- **Research/Exploratory:** Transductive may suffice
- **Risk Management:** Inductive preferred (need to score new cases)

Exam Trap: Transductive is NOT "better" or "worse"; it's "different" for different purposes.

19. Self-Training - Algorithm Overview

Definition:

- "Wrapper" method using supervised classifier iteratively
- Also called "Pseudo-labeling" or "Bootstrapping"
- Uses unlabeled data from a supervised perspective

Full Algorithm:

- 1 Train classifier f on labeled set $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^l$
- 2 Apply f to all unlabeled set $\mathcal{U} = \{x_i\}_{i=l+1}^{l+u}$
- 3 Obtain predicted labels $\hat{y}_i = f(x_i)$ and confidence scores $p_i = P(\hat{y}_i|x_i)$
- 4 Select most confident prediction: $j = \arg \max_{i \in \mathcal{U}} p_i$
- 5 Move $(x_j, \hat{y}_j)$ from $\mathcal{U}$ to $\mathcal{L}$
- 6 Repeat from Step 1 until $\mathcal{U} = \emptyset$ or stopping criterion

Pseudo-label Formalism:

$$\hat{y}_i = \arg \max_{c \in \{1, \dots, C\}} f_c(x_i), \quad \text{with confidence } \max_c f_c(x_i)$$

Stopping Criteria:

- All unlabeled instances labeled
- Confidence below threshold θ
- Maximum iterations reached
- Validation performance plateaus/degrades

20. Self-Training - Mathematical Framework

Objective Function:

$$\min_f \sum_{i=1}^l \ell(f(x_i), y_i) + \lambda \sum_{i=l+1}^{l+u} \ell(f(x_i), \hat{y}_i)$$

where $\hat{y}_i$ are pseudo-labels (dependent on f)

Iterative Optimization:

At iteration t :

$$f^{(t)} = \arg \min_f \sum_{i=1}^{l+m_t} \ell(f(x_i), y_i^{(t)})$$

where $y_i^{(t)}$ are:

- Original labels for $i \leq l$
- Pseudo-labels from iteration $t - 1$ for $l < i \leq l + m_t$

Confidence Selection:

$$(x_j, \hat{y}_j) = \arg \max_{i \in \mathcal{U}} f_{i, \max}^{(t)}(x_i)$$

where $f_{i, \max}(x_i) = \max_c f_c(x_i)$

Batch Selection (k instances):

$$\{(x_j, \hat{y}_j)\}_{j=1}^k = \text{top-}k \text{ instances by } f_{i, \max}(x_i)$$

Reduces computational cost by k times.

21. Self-Training - Advantages

Key Strengths:

1 Algorithm Agnostic:

- Works with ANY supervised classifier: Logistic Regression, SVM, Decision Trees, Neural Networks
- No modification to base classifier needed
- Only requires confidence scores (probabilities)

2 Simplicity:

- Intuitive and easy to implement
- Minimal hyperparameters
- Clear iterative process

3 Flexibility:

- Can be applied to classification and regression
- Can handle multi-class problems
- Can be combined with other techniques

4 Scalability:

- Batch processing (add k instances at a time)
- Can handle large datasets with efficient classifiers
- Parallelizable (train multiple models in parallel)

5 No Feature Split Required:

- Unlike co-training, doesn't need two views
- Can be applied to any feature set

22. Self-Training - Disadvantages & Risks

Major Risks:

1 Error Amplification:

- Initial mistakes with high confidence are added to training set
- These errors propagate and reinforce in subsequent iterations
- Self-reinforcing feedback loop
- **Example:** Model misclassifies a fraud case as normal with 99% confidence, then learns from this error

2 Computational Cost:

- Model retrained each iteration
- If adding 1 instance at a time: u retraining steps
- Batch method: u/k retraining steps (mitigation)

3 Overfitting:

- Model learns its own noise and biases
- Validation performance degrades after early iterations
- Memorizes pseudo-labels rather than generalizing

4 Class Imbalance Amplification:

- Model biased toward majority class
- High-confidence predictions from majority class
- Amplifies imbalance over iterations

23. Self-Training - Mitigation Strategies

Addressing Error Amplification:

1 Confidence Threshold:

- Only add predictions with confidence $> \theta$
- Typical: $\theta = 0.9$ or 0.95
- **Trade-off:** Too high $\Rightarrow$ few labels added; too low $\Rightarrow$ errors introduced

2 Batch Processing:

- Add k most confident instances per iteration
- Reduces computational cost by factor k
- Still risk of adding multiple erroneous labels

3 Validation Set:

- Monitor performance on held-out validation set
- Stop when performance degrades
- Ensures model doesn't overfit to pseudo-labels

4 Co-Training:

- Use two models with different views
- Cross-teaching reduces error amplification
- [See Section on Co-training](#)

5 Ensemble Methods:

- Use multiple models and only add when all agree
- Reduces risk of individual model errors

24. Self-Training - Worked Example (Borrowers)

Dataset: 14 borrowers, 10 labeled, 4 unlabeled

Features: Balance (Feature A), Income (Feature B)

Classifier: Logistic Regression

Step 1: Train on labeled (1-10):

$$P(\text{Default}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \text{Balance} + \beta_2 \text{Income})}}$$

Step 2: Predict unlabeled (11-14):

Borrower	$P(\text{Non-Default})$	$P(\text{Default})$	Prediction
11	1.0000	8.08×10^{-29}	0 (Non-Default)
12	0.9934	6.61×10^{-3}	0
13	0.0000	1.0000	1 (Default)
14	0.9896	1.04×10^{-2}	0

Step 3: Add Borrower 13 (highest confidence: $P = 1.000$) to labeled set

Iterate: Retrain, predict remaining, add next most confident

Final Labels: 11, 12, 14 $\rightarrow$ Non-Default; 13 $\rightarrow$ Default

Key Insight: Near-perfect probabilities suggest overfitting on small sample.

25. Self-Training - Mathematical Example Details

Logistic Regression Setup:

Initial model (first iteration):

$$\log\left(\frac{P}{1-P}\right) = \beta_0 + \beta_1 \text{Balance} + \beta_2 \text{Income}$$

Prediction for Borrower 13:

- Balance = 1.99, Income = -1.04
- $\beta_0 + \beta_1(1.99) + \beta_2(-1.04) \approx 20.3$
- $P(\text{Default}) = \frac{1}{1+e^{-20.3}} = 1.0000$

Why such high confidence?

- Small labeled set (n=10)
- Features are highly predictive in this limited sample
- Model overfits to patterns in the labeled data
- **Risk:** If Borrower 13 is actually Non-Default, error is severe

After Adding Borrower 13:

New model includes Borrower 13 (labeled Default):

- Decision boundary shifts toward Default class
- Borrower 11 now most confident Non-Default
- Pattern: Model reinforces initial high-confidence predictions

Exam Trap: High confidence $\neq$ correct label. Overfitting causes overconfidence.

Definition:

- Inductive method using **two different "views"** of the data
- Two classifiers "teach" each other
- Reduces risk of error amplification (self-training weakness)

Key Requirement:

- Features can be split into **two disjoint and conditionally independent** subsets
- Each view should be sufficient to build a good classifier on its own
- Views are conditionally independent given the class label

Algorithm:

- 1 Split features into Group A and Group B
- 2 Train Model A on Group A (labeled data), Model B on Group B (labeled data)
- 3 Model A predicts labels for all unlabeled using Group A
- 4 Model B predicts labels for all unlabeled using Group B
- 5 Select most confident prediction from Model A → add to Model B's training set
- 6 Select most confident prediction from Model B → add to Model A's training set
- 7 Retrain both models, repeat until no unlabeled data remains

27. Co-Training - Mathematical Framework

Conditional Independence Assumption:

$$P(x_A, x_B | y) = P(x_A | y) \cdot P(x_B | y)$$

where x_A and x_B are the two feature views.

Two Classifiers:

$$f_A : \mathcal{X}_A \rightarrow \mathcal{Y}, \quad f_B : \mathcal{X}_B \rightarrow \mathcal{Y}$$

Cross-Teaching Objective: At iteration t :

$$f_A^{(t)} = \arg \min_{f_A} \sum_{i=1}^{I+m_t^{(A)}} \ell(f_A(x_{i,A}), y_i)$$

$$f_B^{(t)} = \arg \min_{f_B} \sum_{i=1}^{I+m_t^{(B)}} \ell(f_B(x_{i,B}), y_i)$$

where $m_t^{(A)}$ and $m_t^{(B)}$ are pseudo-labels added from the other model.

Confidence Selection:

$$(x_j, \hat{y}_j) = \arg \max_{i \in \mathcal{U}} \max_c f_{A,c}(x_{i,A}) \quad (\text{for Model A's most confident})$$

$$(x_k, \hat{y}_k) = \arg \max_{i \in \mathcal{U}} \max_c f_{B,c}(x_{i,B}) \quad (\text{for Model B's most confident})$$

28. Co-Training - Theoretical Justification

Why Co-Training Works (Theory):

Theorem (Blum & Mitchell, 1998):

- If features are conditionally independent given the class
- AND each view is sufficient for learning
- THEN co-training can learn from unlabeled data
- The two views provide "independent evidence" for labels

Intuitive Explanation:

- Model A makes mistakes on some instances
- Model B makes mistakes on different instances
- When both confidently agree, probability of error is product of individual error probabilities
- Since views are independent: $P(\text{both wrong}) = P(A \text{ wrong}) \cdot P(B \text{ wrong})$
- If each is 90% accurate: $P(\text{both wrong}) = 0.1 \times 0.1 = 0.01$

Error Amplification Reduction:

- Self-training: Model learns from its own errors $\Rightarrow$ amplification
- Co-training: Model learns from other model's evidence $\Rightarrow$ correction
- Mistakes are "filtered" through the other view

29. Co-Training - Advantages & Disadvantages

Advantages:

- ① **Reduced Error Amplification:**
 - Cross-teaching prevents self-reinforcing errors
 - Each model's mistakes are corrected by the other
- ② **Leverages Multiple Views:**
 - Can combine different feature types (text + metadata)
 - Uses complementary information from both views
- ③ **Robustness:**
 - More robust than self-training
 - Less sensitive to initial model errors

Disadvantages:

- ① **Requires Feature Split:**
 - Must find two independent views
 - Not always possible in practice
- ② **Conditional Independence:**
 - Strong assumption
 - Often violated in real data
- ③ **Computational Cost:**
 - Two models $\Rightarrow$ double the cost
 - Still iterative like self-training

30. Co-Training - Worked Example (Borrowers)

Feature Split:

- Group A: Balance (Feature A)
- Group B: Income (Feature B)

Step 1: Train Model A (Balance), Model B (Income)

Model A Predictions:

Borrower	$P(\text{Non-Default})$	$P(\text{Default})$	Prediction
11	1.0000	1.32×10^{-30}	0
12	0.9991	9.13×10^{-4}	0
13	0.0000	1.0000	1
14	0.9983	1.74×10^{-2}	0

Model B Predictions:

Borrower	$P(\text{Non-Default})$	$P(\text{Default})$	Prediction
11	0.9989	1.14×10^{-3}	0
12	0.9635	3.65×10^{-2}	0
13	0.1655	0.8346	1
14	0.9507	4.93×10^{-2}	0

Cross-Teaching:

- Model A most confident: Borrower 13 (Default) → add to Model B's training set
- Model B most confident: Borrower 11 (Non-Default) → add to Model A's training set

31. Co-Training - Feature Split Requirements

Condition 1: Disjoint Features

$$\text{Feature Set } \mathcal{F} = \mathcal{F}_A \cup \mathcal{F}_B, \quad \mathcal{F}_A \cap \mathcal{F}_B = \emptyset$$

Condition 2: Conditional Independence

$$P(x_A, x_B | y) = P(x_A | y) \cdot P(x_B | y)$$

Condition 3: Sufficiency

- Each view alone can build a reasonably good classifier
- $P(y|x_A)$ is informative (not random)
- $P(y|x_B)$ is informative (not random)

Examples of Good Splits:

Domain	View A	View B
Web Pages	Words on page	Anchor text from links
Emails	Subject line words	Body text words
Images	RGB pixels	Edge/HOG features
Video	Audio track	Video frames
Documents	TF-IDF words	Meta-data (author, date)
Medical	Clinical notes	Lab results

32. Co-Training - When It Fails

Reason 1: Views Not Conditionally Independent

- Example: Web page classification where content and anchor text are correlated
- Author writes about same topics in text and links
- **Result:** Models make same errors, no correction

Reason 2: Views Not Sufficient

- Example: One view has very weak predictive power
- Model B (weak view) makes mostly random predictions
- **Result:** Adds noisy labels to Model A, degrades performance

Reason 3: No Natural Feature Split

- Many datasets have highly correlated features
- Example: Financial ratios (all based on accounting data)
- **Result:** Cannot find two independent views

Reason 4: Dataset Shift

- Labeled data distribution differs from unlabeled
- Models learn wrong structure from unlabeled
- **Result:** Co-training fails worse than self-training

33. Unsupervised Pre-Processing - Overview

Definition:

- Use unlabeled data **first**, before supervised learning
- Learn about data structure or create efficient feature representation
- Three main strategies:
 - 1 Feature Extraction (PCA, Autoencoders)
 - 2 Cluster-then-Label
 - 3 Pre-training (Deep Learning)

Core Philosophy:

- Unlabeled data is used to improve **representation**
- Supervised learning uses this improved representation
- Two-stage process: Unsupervised → Supervised

When to Use:

- When unlabeled data is abundant
- When features are high-dimensional or noisy
- When representation learning is critical
- When deep learning is involved (pre-training)

34. Feature Extraction - PCA

Principal Component Analysis (PCA):

Mathematical Formulation:

$$X = U\Sigma V^T \quad (\text{SVD Decomposition})$$

$$X_{\text{PCA}} = XV_k \quad \text{where } V_k \text{ top } k \text{ eigenvectors}$$

Process:

- 1 Compute covariance matrix: $\Sigma = \frac{1}{n}X^T X$
- 2 Find eigenvectors $v_1, \dots, v_D$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D$
- 3 Select top k eigenvectors: $V_k = [v_1, \dots, v_k]$
- 4 Project data: $Z = XV_k$ (reduced dimensions)
- 5 Train supervised classifier on Z using labeled data

Advantages:

- Reduces dimensionality (helps with overfitting)
- Captures maximum variance in data
- Unsupervised (doesn't use labels)

Disadvantages:

- **Linear:** Cannot capture non-linear structure
- **Variance-focused:** May discard discriminative features with low variance

35. Feature Extraction - Autoencoders

Autoencoder Architecture:

$$h = \sigma_e(W_e x + b_e) \quad (\text{Encoder})$$

$$\hat{x} = \sigma_d(W_d h + b_d) \quad (\text{Decoder})$$

Objective: Minimize reconstruction error:

$$\mathcal{L}_{AE} = \|x - \hat{x}\|^2 = \|x - \sigma_d(W_d \sigma_e(W_e x + b_e) + b_d)\|^2$$

Process:

- 1 Train autoencoder on ALL data (labeled + unlabeled)
- 2 Encoder learns compressed representation h for each x
- 3 h captures essential features in lower dimension ($d \ll D$)
- 4 Use h as features for supervised classifier on labeled data

Advantages over PCA:

- **Non-linear:** Can capture complex structure
- **Flexible:** Can be deeper, more expressive
- **Learns features:** Not just variance, but reconstruction

Disadvantages:

- **Computational:** More expensive than PCA
- **Hyperparameter Tuning:** Architecture, learning rate, etc.

36. Cluster-then-Label - Detailed

Two-Stage Process:

Stage 1: Clustering (Unsupervised)

- Apply clustering algorithm (e.g., K-means) to ALL data
- Partition data into k clusters: $C_1, \dots, C_k$
- Based on feature similarity, NOT labels

K-means Objective:

$$\min_{\mu_1, \dots, \mu_k} \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Stage 2: Label Assignment

- For each cluster C_j , find labeled points $\mathcal{L}_j \subset C_j$
- Assign cluster label: $\hat{y}_j = \text{majority}(\{y_i : i \in \mathcal{L}_j\})$
- If no labels in cluster: unassigned (or nearest cluster)
- All unlabeled points in cluster get $\hat{y}_j$

Final Step:

- Train supervised classifier on fully labeled dataset
- Pseudo-labels become training data

Limitation:

- Relies on Clustering Assumption
- If clusters don't correspond to classes, all pseudo-labels wrong

37. Cluster-then-Label - Advantages & Risks

Advantages:

- 1 **Computational Efficiency:**
 - Single clustering pass ($O(nkd)$ iterations)
 - No iterative model retraining
 - Much faster than self-training/co-training
- 2 **Simplicity:**
 - Easy to understand and implement
 - Few hyperparameters (k , distance metric)
- 3 **Scalable:**
 - Works with large datasets
 - K-means scales $O(n)$ with efficient implementations

Risks:

- 1 **Assumption Dependency:**
 - Requires Clustering Assumption to be strongly satisfied
 - If violated, method fails completely
- 2 **Propagation of Errors:**
 - One mislabeled cluster affects all points in it
 - No mechanism to correct mistakes
- 3 **Choice of k :**
 - Must know number of clusters (classes)
 - Wrong k leads to poor results

38. Pre-training (Deep Learning)

Definition:

- Use unlabeled data to **initialize** neural network weights
- Two-phase approach: Pre-training + Fine-tuning

Phase 1: Unsupervised Pre-training

- Train autoencoder or generative model on unlabeled data
- Objective: Learn good feature representations
- Output: Initialized weights for neural network

Phase 2: Supervised Fine-tuning

- Use labeled data to fine-tune the network
- Adjust weights for classification task
- Typically uses smaller learning rate

Mathematical Framework:

Pre-training:

$$\theta_{\text{pre}} = \arg \min_{\theta} \sum_{i=l+1}^{l+u} \|x_i - \hat{x}_i(x_i; \theta)\|^2$$

Fine-tuning:

$$\theta_{\text{final}} = \arg \min_{\theta} \sum_{i=1}^l \ell(f(x_i; \theta), y_i) + \lambda \|\theta - \theta_{\text{pre}}\|^2$$

Key Insight: Pre-training gives model a "head start" in understanding data structure.

39. Pre-training - Benefits & Limitations

Benefits:

1 Better Generalization:

- Pre-training acts as regularization
- Reduces overfitting with limited labels

2 Faster Convergence:

- Starts from good initial weights
- Fine-tuning requires fewer iterations

3 Leverages Unlabeled Data:

- Massive unlabeled datasets can be used
- Learns rich feature representations

4 State-of-the-art:

- Used in modern deep learning (BERT, GPT, etc.)
- Essential for low-label scenarios

Limitations:

1 Computational Cost:

- Pre-training on large datasets is expensive
- Requires significant GPU resources

2 Architecture Design:

- Autoencoder must be designed carefully
- Too simple $\Rightarrow$ poor representation
- Too complex $\Rightarrow$ overfitting

3 Transfer Mismatch:

- Pre-training distribution may differ from task
- Fine-tuning may not overcome mismatch

40. Comparison of Pre-Processing Methods

Comparison Table:

Aspect	PCA	Autoencoder	Cluster-then-Label
Type	Linear	Non-linear	Two-stage
Use of Unlabeled	All data	All data	All data
Output	Low-dim features	Low-dim features	Pseudo-labels
Computational Cost	Low	Medium-High	Low
Assumptions	Linear structure	Architecture design	Clustering
Scalability	Good	Moderate	Good
Interpretability	High	Low	Medium
Risk	Discards info	Overfitting	Assumption violation

When to Use Each:

- **PCA:** Linear data, need speed and interpretability
- **Autoencoder:** Non-linear data, deep learning pipeline
- **Cluster-then-Label:** Strong clustering, computationally efficient

41. Exam Trap 1: Assumptions are Optional

The Misconception:

- "Semi-supervised learning always improves performance because it uses more data"
- "The assumptions are just theoretical, they don't matter in practice"

The Reality:

- **FALSE!** SSL only works if assumptions are satisfied
- If assumptions violated, SSL can perform **worse** than supervised-only

Example:

- Supervised-only with 100 labels: 70% accuracy
- SSL with 100 labels + 10,000 unlabeled: 55% accuracy
- Why? Unlabeled data misleads the model

Key Takeaway:

- Always check assumptions before applying SSL
- Test with small experiments before full deployment
- Monitor performance degradation as iterations progress

Exam Expectation: You must know that SSL requires assumptions and can fail.

42. Exam Trap 2: Transductive vs Inductive Confusion

The Misconception:

- "Transductive methods are just a type of inductive method"
- "Both create models that can predict new data"

The Reality:

- **FALSE!** They are fundamentally different
- Transductive $\neq$ Inductive

Key Differences to Remember:

Transductive	Inductive
No model created	Model created
Only current data	Generalizes to new data
Cannot deploy	Deployable
Offline analysis	Production-ready

Example:

- Label Propagation: Transductive (no model)
- Self-training: Inductive (creates model)

Exam Expectation: You must be able to classify methods correctly and choose appropriate for scenario.

43. Exam Trap 3: High Confidence = Correct

The Misconception:

- "If the model is 99% confident, the prediction must be correct"
- "High confidence pseudo-labels are always reliable"

The Reality:

- **FALSE!** Confidence not equal Correctness
- Overconfident models make mistakes with high confidence

Why This Happens:

- 1 **Small Labeled Set:** Model overfits to few examples
- 2 **Class Imbalance:** Model is confident about majority class
- 3 **Noisy Features:** Spurious correlations in labeled data

Example (Borrowers):

- Borrower 13 predicted Default with $P = 1.0000$
- This was based on only 10 labeled examples
- If label was wrong, error would be severe

Prevention:

- Use validation set to monitor confidence calibration
- Use confidence thresholds
- Consider ensemble methods

44. Exam Trap 4: Error Amplification

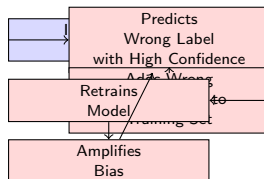
The Misconception:

- "Adding pseudo-labels always helps because it adds more training data"
- "Self-training will eventually converge to the correct labels"

The Reality:

- **FALSE!** Self-training amplifies errors
- Once an error is introduced, it reinforces itself

The Amplification Cycle:



Solution:

- Co-training (cross-teaching)
- Confidence thresholds
- Ensemble methods

45. Exam Trap 5: More Data Always Better

The Misconception:

- "More data is always better for machine learning"
- "Unlabeled data can only help, never hurt"

The Reality:

- **FALSE!** More data can hurt if:
 - ① Data is from different distribution
 - ② Assumptions are violated
 - ③ Data is noisy

When Unlabeled Data Hurts:

Situation	Why It Hurts
Dataset Shift	Structure doesn't match classification
Violated Assumptions	Unlabeled data misleads model
Noisy Unlabeled	Learns incorrect structure
Too Many Features	Manifold assumption weakens

Example:

- Supervised: 70% accuracy with 100 labels
- SSL with 100 labels + 10,000 unlabeled from different distribution: 50% accuracy

Key Insight: Quality and relevance $\neq$ Quantity

46. Exam Trap 6: Co-training Always Better

The Misconception:

- "Co-training is always better than self-training"
- "Since co-training uses two views, it's strictly superior"

The Reality:

- **FALSE!** Co-training requires strong assumptions
- If assumptions violated, self-training may be better

When Co-training Fails:

- 1 **No Natural Feature Split:**
 - Most datasets don't have two independent views
 - Forced split reduces performance
- 2 **Views Not Independent:**
 - Correlated views don't provide independent evidence
 - Models make same errors
- 3 **Views Not Sufficient:**
 - One view is weak or random
 - Adds noise rather than signal

When Self-training Might Be Better:

- No clear feature split exists
- Features are already well-engineered
- Simplicity is preferred

47. Exam Trap 7: Pseudo-labels = True Labels

The Misconception:

- "After SSL, the pseudo-labels are as good as true labels"
- "We can treat pseudo-labeled data the same as labeled data"

The Reality:

- **FALSE!** Pseudo-labels are estimates
- They contain errors, biases, and uncertainties

Key Differences:

True Labels	Pseudo-Labels
Ground truth	Estimated
No errors (ideally)	Contains errors
Reliable	Uncertain
Independent	Model-dependent
Can be trusted	Should be questioned

Consequences:

- Treating pseudo-labels as truth leads to overconfidence
- Errors are underestimated
- Model calibration suffers

Best Practices:

- Use pseudo-labels only as additional data
- Continue to rely on true labels for evaluation
- Monitor performance degradation

48. Exam Trap Summary

Seven Key Exam Traps to Remember:

- ❶ **Trap 1: Assumptions are Optional**
Reality: SSL fails without proper assumptions
- ❷ **Trap 2: Transductive = Inductive**
Reality: Fundamentally different; know the distinction
- ❸ **Trap 3: High Confidence = Correct**
Reality: Overconfidence leads to error amplification
- ❹ **Trap 4: Error Amplification is Minor**
Reality: Self-training can severely degrade performance
- ❺ **Trap 5: More Data Always Better**
Reality: Unlabeled data can hurt if assumptions violated
- ❻ **Trap 6: Co-training Always Better**
Reality: Requires strong assumptions; can fail
- ❼ **Trap 7: Pseudo-labels = True Labels**
Reality: Pseudo-labels are estimates with errors

Key Advice: Always question assumptions, monitor performance, and validate results.

General SSL Objective:

$$\mathcal{L}(\theta) = \underbrace{\sum_{i=1}^l \ell(f_{\theta}(x_i), y_i)}_{\text{Supervised Loss}} + \lambda \underbrace{\sum_{i=l+1}^{l+u} \mathcal{R}_{\text{unsup}}(f_{\theta}, x_i)}_{\text{Unsupervised Regularizer}}$$

Common Unsupervised Regularizers:

1. **Low-Density Separation:**

$$\mathcal{R}_{\text{LDS}}(f, x) = -\max_c f_c(x) \quad (\text{Minimize entropy})$$

or

$$\mathcal{R}_{\text{LDS}}(f, x) = \sum_{c=1}^C f_c(x) \log f_c(x) \quad (\text{Entropy})$$

2. **Manifold Regularization:**

$$\mathcal{R}_{\text{M}}(f, x) = \sum_{i,j} w_{ij} \|f(x_i) - f(x_j)\|^2$$

where w_{ij} are similarity weights on the manifold

3. **Cluster Assumption:**

$$\mathcal{R}_{\text{C}}(f, x) = \sum_{i=l+1}^{l+u} \mathbb{I}[\text{cluster}(x_i) \neq \arg \max_c f_c(x_i)]$$

Iterative Objective:

At iteration t :

$$\theta^{(t)} = \arg \min_{\theta} \sum_{i=1}^{l+m_t} \ell(f_{\theta}(x_i), y_i^{(t)})$$

where:

$$y_i^{(t)} = \begin{cases} y_i & \text{if } i \leq l \text{ (original labels)} \\ \hat{y}_i^{(t-1)} & \text{if } l < i \leq l + m_t \text{ (pseudo-labels from previous iteration)} \end{cases}$$

Confidence Selection:

$$x_j^{(t)} = \arg \max_{i \in \mathcal{U}_t} f_{\theta^{(t)}, \max}(x_i)$$

where $f_{\theta, \max}(x_i) = \max_{c \in \{1, \dots, C\}} f_{\theta, c}(x_i)$

Pseudo-label for selected instance:

$$\hat{y}_j^{(t)} = \arg \max_c f_{\theta^{(t)}, c}(x_j)$$

Batch Version (add k instances):

$$\mathcal{S}_t = \{(x_j, \hat{y}_j^{(t)}) : j \text{ are top-}k \text{ in } f_{\theta^{(t)}, \max}(x_i)\}$$

Stopping Condition:

$$\text{Stop if } \mathcal{U}_t = \emptyset \text{ or } \text{ValLoss}(\theta^{(t)}) > \text{ValLoss}(\theta^{(t-1)})$$

51. Co-Training - Mathematical Details

Two Models:

$$f_A : \mathcal{X}_A \rightarrow \Delta^C, \quad f_B : \mathcal{X}_B \rightarrow \Delta^C$$

Training Objectives:

$$\theta_A^{(t)} = \arg \min_{\theta_A} \sum_{i=1}^{l+m_t^{(A)}} \ell(f_A(x_{i,A}; \theta_A), y_i)$$

$$\theta_B^{(t)} = \arg \min_{\theta_B} \sum_{i=1}^{l+m_t^{(B)}} \ell(f_B(x_{i,B}; \theta_B), y_i)$$

Cross-Teaching Selection:

$$x_{j_A}^{(t)} = \arg \max_{i \in \mathcal{U}_t} f_{A, \max}(x_{i,A}; \theta_A^{(t)})$$

$$x_{j_B}^{(t)} = \arg \max_{i \in \mathcal{U}_t} f_{B, \max}(x_{i,B}; \theta_B^{(t)})$$

Exchange Pseudo-labels:

- Add $(x_{j_A}, \hat{y}_{j_A}^{(t)})$ to Model B's training set
- Add $(x_{j_B}, \hat{y}_{j_B}^{(t)})$ to Model A's training set

Conditional Independence Assumption:

$$P(x_A, x_B | y) = P(x_A | y) \cdot P(x_B | y)$$

Error Probability (if both agree):

$$P(\text{both wrong}) = P(A \text{ wrong}) \cdot P(B \text{ wrong}) \quad (\text{if independent})$$

52. Label Propagation - Mathematical Details

Graph Construction:

$$w_{ij} = \exp \left(- \frac{\|x_i - x_j\|^2}{2\sigma^2} \right)$$

Normalized Weights:

$$P_{ij} = \frac{w_{ij}}{\sum_{k=1}^n w_{ik}}$$

Label Propagation Update:

$$Y^{(t+1)} = PY^{(t)}$$

where $Y \in \mathbb{R}^{n \times C}$ with:

$$Y_{i,c} = \begin{cases} 1 & \text{if } y_i = c \text{ and } i \text{ is labeled} \\ 0 & \text{otherwise} \end{cases}$$

Clamping Labeled Nodes:

$$Y_{i,:}^{(t+1)} = Y_{i,:}^{(0)} \quad \text{for labeled nodes } i = 1, \dots, l$$

Fixed Point:

$$Y^* = \lim_{t \rightarrow \infty} Y^{(t)}$$

Final Classification:

$$\hat{y}_i = \arg \max_c Y_{i,c}^* \quad \text{for unlabeled } i = l+1, \dots, n$$

Alternative Formulation (Laplacian):

$$Y^* = \arg \min_Y \text{Trace}(Y^T L Y) \quad \text{s.t. } Y_{i,:} = Y_{i,:}^{(0)} \text{ for labeled}$$

where $L = D - W$ is the graph Laplacian.

53. Unsupervised Pre-processing - Mathematical Details

PCA:

$$\Sigma = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})^T$$

$$\Sigma V = V \Lambda \quad (\text{Eigen-decomposition})$$

$$Z_i = V_k^T (x_i - \bar{x}) \quad \text{where } V_k \text{ top } k \text{ eigenvectors}$$

Autoencoder:

$$h_i = \sigma_e(W_e x_i + b_e) \quad (\text{Encoder})$$

$$\hat{x}_i = \sigma_d(W_d h_i + b_d) \quad (\text{Decoder})$$

$$\mathcal{L}_{\text{AE}} = \frac{1}{n} \sum_{i=1}^n \|x_i - \hat{x}_i\|^2$$

K-means (Cluster-then-Label):

$$\mu_j^{(t+1)} = \frac{1}{|\mathcal{C}_j^{(t)}|} \sum_{x_i \in \mathcal{C}_j^{(t)}} x_i$$

$$\mathcal{C}_j^{(t)} = \{x_i : \|x_i - \mu_j^{(t)}\|^2 \leq \|x_i - \mu_k^{(t)}\|^2 \forall k\}$$

Cluster Label Assignment:

$$\hat{y}_j = \arg \max_c \sum_{i \in \mathcal{C}_j \cap \mathcal{L}} \mathbb{I}[y_i = c]$$

$$\hat{y}_i = \hat{y}_j \quad \text{for all } i \in \mathcal{C}_j \setminus \mathcal{L}$$

54. Key Mathematical Results Summary

Performance Bounds:

- SSL can reduce sample complexity from $O(I)$ to $O(\log I)$ under manifold assumptions
- Error rate: $\epsilon_{\text{SSL}} \leq \epsilon_{\text{supervised}} + O\left(\frac{1}{\sqrt{u}}\right)$

Regularization Interpretation:

$$\text{SSL} \iff \text{Supervised} + \text{Regularization from } P(x)$$

Manifold Learning Connection:

$$\text{SSL on manifold} \iff \text{Laplacian Regularization}$$

$$\mathcal{R}(f) = \int_{\mathcal{M}} \|\nabla_{\mathcal{M}} f\|^2 dP(x)$$

Information Theory:

$$I(x, y) = I(x_A, y) + I(x_B, y) \quad (\text{co-training with independent views})$$

Key Inequalities:

- ① **Clustering + Smoothness**: $\Rightarrow$ Low-density separation reduces VC-dimension
- ② **Manifold**: $\Rightarrow$ Effective dimension reduces from D to d
- ③ **Co-training**: $\Rightarrow$ Error $\propto \epsilon_A \cdot \epsilon_B$ (product of errors)

55. Summary - What SSL Is and Isn't

What SSL IS:

- A paradigm for learning with both labeled and unlabeled data
- A way to leverage cheap unlabeled data to improve generalization
- A set of techniques: Self-training, Co-training, Unsupervised Pre-processing
- A framework based on three key assumptions

What SSL IS NOT:

- A replacement for supervised learning
- A guarantee of improved performance
- A magic solution for all problems
- A way to use data from different distributions

Core Insight:

- Unlabeled data reveals structure $P(x)$
- This structure guides classification $P(y|x)$
- If structure doesn't map to classes, SSL fails

56. Key Takeaways - Assumptions

Three Core Assumptions:

1 Clustering Assumption:

- Data forms distinct clusters
- Each cluster corresponds to one class
- Boundaries in low-density regions

2 Smoothness (Continuity) Assumption:

- Close points $\Rightarrow$ same label
- Decision boundary in low-density regions
- Local consistency

3 Manifold Assumption:

- High-dimensional data on low-dimensional manifold
- Labels vary smoothly on manifold
- Unlabeled data maps the manifold

Critical: All three must be satisfied for SSL to work effectively.

Self-Training:

- Simple, intuitive wrapper method
- Works with any supervised classifier
- **Risk:** Error amplification, computational cost, overfitting
- **Best for:** When no natural feature split exists

Co-Training:

- Uses two independent views
- Reduces error amplification through cross-teaching
- **Risk:** Requires conditional independence of views
- **Best for:** When natural feature split exists (e.g., text + metadata)

Unsupervised Pre-processing:

- Feature extraction (PCA, Autoencoders)
- Cluster-then-Label
- Pre-training (Deep Learning)
- **Best for:** High-dimensional data, representation learning

58. Key Takeaways - Inductive vs Transductive

Inductive Methods:

- Build generalizable model $f : X \rightarrow Y$
- Can classify new, unseen data
- **Examples:** Self-training, Co-training
- **Use when:** Need production model, future predictions

Transductive Methods:

- Only label current unlabeled data
- No model created
- **Examples:** Label Propagation
- **Use when:** Offline analysis only, no future predictions

Decision Framework:

Requirement	Inductive	Transductive
Future predictions	✓	×
Deployment	✓	×
Current dataset only	×	✓

59. Practical Implementation Guidelines

Before Applying SSL:

① Check Assumptions:

- Do data form clusters?
- Are boundaries in low-density regions?
- Does data lie on a manifold?

② Verify Distribution:

- Labeled and unlabeled data from same distribution?
- No dataset shift?

③ Start Simple:

- Begin with supervised baseline
- Compare SSL performance
- Monitor degradation

During SSL Training:

① Use Validation Set:

- Monitor performance after each iteration
- Stop when performance degrades

② Set Confidence Threshold:

- Only add high-confidence predictions
- Typical threshold: 0.9-0.95

③ Batch Processing:

- Add k instances per iteration
- Reduces computational cost

60. Final Summary - Key Points for Exam

Remember These 10 Points:

- 1 **SSL** is used when labeled data is scarce, unlabeled is abundant
- 2 **Three Assumptions:** Clustering, Smoothness, Manifold
- 3 **Assumptions MUST be satisfied** for SSL to work
- 4 **Self-training** is simple but risks error amplification
- 5 **Co-training** uses two views to reduce error amplification
- 6 **Inductive** methods build models; **Transductive** only label current data
- 7 **Feature Extraction** (PCA, Autoencoders) uses unlabeled for representation
- 8 **Cluster-then-Label** uses clustering then majority voting
- 9 **High confidence $\neq$ Correct** - overconfidence is a trap
- 10 **More data is NOT always better** - data quality and relevance matter

Final Advice: Always validate SSL performance against supervised baseline. Monitor assumptions. Be aware of failure modes.

Exam Ready!